

Sistem Programlama Projesi

Çok Threadli Dosya Kopyalama ve Tek Yönlü Senkronizasyon Aracı

Grup No: 19

SEVİLAY ÇOLAKER (170519004)

YUSUF TARLAN (150619027)

ZEYNEP SUDE CAN (100619047)

Rapor Tarihi: 2 Haziran 2026

İçindekiler

Projenin Amacı	3
Proje Dosya Dizininin Açıklaması	3
Projede Kullanılan Algoritma ve Akış Şeması	4
Temel Veri Yapıları	4
Dosya Karar Algoritması	4
Akış Şeması	5
Kritik ve Önemli Kodların Açıklaması	5
main.c	5
scanner.c	6
queue.c	6
worker.c	6
log.c	7
Genel Değerlendirme	7

Projenin Amacı

Bu projenin amacı, C dili ve POSIX threads (`pthread`) kullanılarak çok threadli çalışan bir dosya kopyalama ve tek yönlü senkronizasyon aracı geliştirmektir. Program, kaynak dizindeki dosyaları hedef dizine aktarır; hedefte bulunmayan dosyaları kopyalar, hedefte bulunup değiştiği anlaşılan dosyaları ise günceller.

Burada “tek yönlü senkronizasyon” ifadesi önemlidir. Program yalnızca kaynak dizinden hedef dizine doğru çalışır. Kaynak dizinde silinen bir dosya hedef dizinden otomatik olarak silinmez. Yani araç, iki klasörü tamamen aynı hale getiren tam bir ayna sistemi değil; kaynak taraftaki eksik veya değişmiş içeriği hedef tarafa taşıyan bir kopyalama/güncelleme aracıdır.

Proje sistem programlama açısından birkaç temel konuyu uygulamalı olarak gösterir:

- Thread oluşturma ve yönetme
- Mutex ile ortak veriyi koruma
- Condition variable ile thread’leri bekletme ve uyandırma
- POSIX dosya sistem çağrılılarıyla dosya/dizin işlemleri yapma
- Hata durumlarında loglama ve program akışını kontrollü sürdürme

Basit bir benzetmeyle, proje bir depo taşıma sistemi gibi düşünülebilir. Scanner thread depoda dolaşp taşınacak kutuları listeye ekleyen kişidir. Worker thread’ler ise listedeki kutuları alıp hedef depoya taşıyan işçilerdir. Ortak iş listesi ise thread-safe kuyruktur.

Proje Dosya Dizininin Açıklaması

Proje, kaynak kodları, başlık dosyaları, test dizinleri ve yardımcı betikleri ayrı tutacak şekilde düzenlenmiştir. Bu yapı kodun okunmasını ve sorumlulukların ayrılmasını kolaylaştırır.

```
.
├── Makefile
├── README.md
├── benchmark.sh
├── include/
│   ├── common.h
│   ├── queue.h
│   ├── scanner.h
│   ├── worker.h
│   └── log.h
├── src/
│   ├── main.c
│   ├── queue.c
│   ├── scanner.c
│   ├── worker.c
│   └── log.c
├── source_dir/
└── dest_dir/
```

Makefile, projenin derleme komutlarını içerir. `gcc`, `-Wall`, `-Wextra` ve `-pthread` seçenekleriyle kaynak dosyaları derleyerek `copy_tool` çalıştırılabilir dosyasını üretir.

`README.md`, projenin kullanımını, mimarisini, özelliklerini ve sınırlamalarını açıklayan kullanıcı dokümanıdır.

`benchmark.sh`, farklı worker thread sayılarıyla performans karşılaştırması yapmak için hazırlanmış yardımcı betiktir. Tek thread ve çok threadli çalışma sürelerini kıyaslamak için kullanılır.

`include/` dizini başlık dosyalarını içerir:

- `common.h`: Ortak kullanılan veri yapılarını ve sistem başlıklarını içerir. `CopyTask` ve `DirNode` yapıları burada tanımlanır.
- `queue.h`: Dosya kopyalama kuyruğu olan `TaskQueue` ve dizin gezme kuyruğu olan `DirQueue` yapılarını tanımlar.
- `scanner.h`: Kaynak dizini tarayan `scan_directory` fonksiyonunun bildirimini içerir.
- `worker.h`: Worker thread fonksiyonunun bildirimini içerir.
- `log.h`: Thread-safe loglama fonksiyonlarının bildirimlerini içerir.

`src/` dizini gerçek uygulama kodlarını içerir:

- `main.c`: Programın başlangıç noktasıdır. Argümanları kontrol eder, dizinleri doğrular, worker thread'leri başlatır ve işlem sonunda kaynakları temizler.
- `scanner.c`: Kaynak dizini recursive olarak tarar ve kopyalanması/güncellenmesi gereken dosyaları kuyruğa ekler.
- `queue.c`: Thread-safe dosya iş kuyruğunu ve dizin gezme kuyruğunu yönetir.
- `worker.c`: Kuyruktan iş alır ve dosyaları güvenli şekilde hedef dizine kopyalar.
- `log.c`: Log dosyasına thread-safe şekilde kayıt yazar.

`source_dir/`, örnek kaynak dizindir. Program çalıştırılırken kaynak olarak verilebilir. `dest_dir/`, örnek hedef dizindir. Programın ürettiği veya güncellediği dosyalar burada gözlemlenebilir.

Projede Kullanılan Algoritma ve Akış Şeması

Projede temel olarak üretici-tüketici algoritması kullanılır. Üretici taraf scanner'dır; kaynak dizini gezer ve yapılacak dosya işlerini kuyruğa ekler. Tüketici taraf worker thread'lerdir; her worker kuyruktan bir `CopyTask` alır ve ilgili dosyayı hedef dizine kopyalar.

Bu yapı tek threadli kopyalamaya göre daha verimli olabilir. Çünkü bir worker dosya yazarken başka bir worker başka dosyayı okuyabilir veya yazabilir. Özellikle çok sayıda dosya olduğunda işler paralel dağıtılır.

Temel Veri Yapıları

`CopyTask`, bir dosya kopyalama işini temsil eder. Kaynak dosya yolu, hedef dosya yolu ve işlemin yeni kopyalama mı yoksa güncelleme mi olduğunu belirten `is_update` alanını içerir.

`TaskQueue`, worker thread'lerin ortak kullandığı dosya iş kuyruğudur. Sabit kapasitesi `QUEUE_SIZE = 512` olarak belirlenmiştir. Kuyruk, `pthread_mutex_t` ile korunur. Kuyruk boşken worker thread'ler `not_empty` condition variable üzerinde bekler. Kuyruk doluyken scanner thread `not_full` condition variable üzerinde bekler.

`DirQueue`, scanner'ın recursive dizin taramasını yönetmek için kullandığı ayrı bir kuyruktur. Scanner bir alt dizin gördüğünde bu dizini `DirQueue` içine ekler ve daha sonra sırayla işler.

Dosya Karar Algoritması

Scanner her düzenli dosya için şu kararları verir:

1. Hedefte dosya yoksa dosya `COPY` işi olarak kuyruğa eklenir.
2. Hedefte dosya varsa ve kaynak dosyanın değiştirilme zamanı daha yeniyse dosya `UPDATE` işi olarak kuyruğa eklenir.
3. Kaynak ve hedef dosya boyutları farklıysa dosya `UPDATE` işi olarak kuyruğa eklenir.
4. Zaman ve boyut aynı görünse bile dosya içerikleri blok blok karşılaştırılır. İçerik farklıysa dosya yine `UPDATE` olarak kuyruğa eklenir.

5. Dosya aynıysa herhangi bir işlem yapılmaz.

Bu yaklaşım hızlı kontrolleri önce kullanır. Yani önce dosya var mı, boyut değişmiş mi, zaman bilgisi daha yeni mi diye bakılır. Daha pahalı olan içerik karşılaştırması yalnızca gerekli durumda yapılır.

Akış Şeması

Başla

↓

Komut satırı argümanlarını kontrol et

↓

Thread sayısını doğrula

↓

Kaynak ve hedef dizinleri gerçek yollarıyla doğrula

↓

Hedef kaynak dizinin içinde mi kontrol et

↓

TaskQueue yapısını başlat

↓

Worker thread'leri oluştur

↓

Scanner kaynak dizini tarar

↓

Alt dizinler DirQueue içine eklenir

↓

Kopyalanacak/güncellenecek dosyalar TaskQueue içine eklenir

↓

Worker thread'ler kuyruktan iş alır

↓

Dosya geçici dosyaya blok blok kopyalanır

↓

Kopyalama başarılıysa geçici dosya rename ile hedefe taşınır

↓

Scanner bittiğinde shutdown bayrağı ayarlanır

↓

Worker thread'lerin bitmesi beklenir

↓

Kaynaklar temizlenir ve log kapatılır

↓

Bitir

Kritik ve Önemli Kodların Açıklaması

main.c

main.c, programın kontrol merkezidir. İlk olarak argüman sayısını ve thread sayısını kontrol eder. Kullanıcı programı şu formatta çalıştırmalıdır:

```
./copy_tool <thread_sayisi> <kaynak_dizin> <hedef_dizin>
```

Bu dosyadaki önemli noktalardan biri kaynak ve hedef dizin doğrulamasıdır. `prepare_directories` fonksiyonu kaynak dizinin gerçekten var olup olmadığını, hedefin dizin olup olmadığını ve hedef dizinin kaynak dizinin içinde kalıp kalmadığını kontrol eder.

Hedef dizinin kaynak dizinin içinde olması tehlikelidir. Çünkü scanner kaynak dizini gezerken hedef dizini de kaynak içeriği gibi görebilir. Bu durumda `backup/backup/backup` şeklinde sonsuz

büyüyen bir kopyalama davranışı oluşabilir. Bu nedenle `is_same_or_child_path` fonksiyonu ile bu durum baştan engellenir.

Argümanlar ve dizinler doğrulandıktan sonra `queue_init` çağrılır, worker thread'ler oluşturulur ve `scan_directory` fonksiyonu çalıştırılır. Scanner işi bitirdiğinde `shutdown` bayrağı ayarlanır ve bekleyen worker thread'ler uyandırılır. Son olarak tüm thread'ler `pthread_join` ile beklenir.

scanner.c

`scanner.c`, kaynak dizinin içeriğini gezen modüldür. Bu dosya proje için üretici rolündedir. `scan_directory` fonksiyonu önce kök kaynak ve hedef dizini `DirQueue` içine ekler. Daha sonra kuyruktan dizin çekerek bu dizindeki dosya ve alt dizinleri inceler.

Scanner sembolik linkleri bilinçli olarak atlar. Bunun nedeni, sembolik linklerin takip edilmesi halinde beklenmeyen dizin döngüleri veya kaynak dışına taşan kopyalama davranışları oluşabilmesidir.

Alt dizin görüldüğünde hedef tarafta karşılığı oluşturulur. Eğer hedefte aynı isimde bir şey varsa ama bu bir dizin değilse, scanner o alt ağacı atlar ve hata loglar. Bu davranış, hatanın büyüyerek worker tarafında daha karmaşık sorunlara dönüşmesini engeller.

`files_differ` fonksiyonu, aynı boyut ve aynı zaman bilgisine sahip görünen dosyaları blok blok karşılaştırır. Böylece yalnızca `mtime` ve boyuta bakıldığı için kaçabilecek içerik değişiklikleri de yakalanır.

queue.c

`queue.c`, projenin thread senkronizasyonu açısından en kritik dosyalarından biridir. `TaskQueue`, birden fazla worker thread tarafından ortak kullanıldığı için mutex ile korunur.

`queue_push`, scanner'ın kuyruğa yeni iş eklediği fonksiyondur. Kuyruk doluysa scanner `not_full` condition variable üzerinde bekler. Kuyrukta yer açılınca worker thread'lerden biri `not_full` sinyali gönderir ve scanner kaldığı yerden devam eder.

`queue_pop`, worker thread'lerin kuyruktan iş aldığı fonksiyondur. Kuyruk boşsa worker thread `not_empty` üzerinde bekler. Scanner yeni iş eklediğinde `not_empty` sinyali gönderir ve worker uyanır.

`shutdown` bayrağı, scanner'ın artık yeni iş üretmeyeceğini belirtir. Kuyruk boş ve `shutdown` aktifse worker thread döngüden çıkar. Bu mekanizma olmazsa worker thread'ler boş kuyrukta sonsuza kadar bekleyebilir.

worker.c

`worker.c`, gerçek dosya kopyalama işlemini yapan modüldür. Her worker thread aynı fonksiyonu çalıştırır: `worker_thread`. Bu fonksiyon sürekli olarak `queue_pop` ile yeni iş almaya çalışır.

Dosya kopyalama işleminde doğrudan hedef dosyayı sıfırlayıp yazmak yerine geçici dosya kullanılır. `create_temp_file`, hedef dosyanın yanında benzersiz bir geçici dosya oluşturur. Worker önce kaynak dosyayı bu geçici dosyaya 4096 baytlık bloklarla yazar.

Kopyalama tamamlanınca `fsync` ile verinin diske yazılması istenir. Ardından geçici dosya kapatılır ve `rename` ile hedef dosyanın yerine geçirilir. Bu atomik güncelleme yaklaşımı önemlidir. Çünkü yazma sırasında hata olursa eski hedef dosya korunur. Doğrudan `O_TRUNC` ile hedefi açmak eski dosyayı baştan boşaltacağı için veri kaybına yol açabilirdi.

Worker ayrıca kaynak dosyanın izinlerini hedef dosyaya uygulamaya çalışır. Bu işlem başarısız olursa kopyalama tamamen iptal edilmez; hata loglanır.

log.c

`log.c`, programdaki olayları `copy_tool.log` dosyasına yazar. Log satırlarında zaman bilgisi, thread ID ve olay tipi bulunur. Örneğin `COPY`, `UPDATE`, `ERROR` ve `INFO` gibi olaylar kaydedilir.

Birden fazla thread aynı anda log yazabileceği için `log_event` fonksiyonu mutex kullanır. Böylece iki thread'in log satırları birbirine karışmaz. Bu küçük gibi görünen ayrıntı, çok threadli programlarda hata ayıklama açısından önemlidir.

Genel Değerlendirme

Proje, klasik bir dosya kopyalama aracından daha fazlasını gösterir. Thread'ler arası iş paylaşımı, kuyruk tabanlı üretici-tüketici modeli, POSIX dosya işlemleri, hata yönetimi ve güvenli dosya güncelleme gibi sistem programlama konularını bir araya getirir.

Kodun önemli tarafı yalnızca dosya kopyalaması değildir. Asıl öğrenme noktası, birden fazla thread'in aynı iş kuyruğunu güvenli biçimde paylaşması ve dosya sistemi üzerinde yarım kalmış işlem bırakmadan çalışmasıdır.